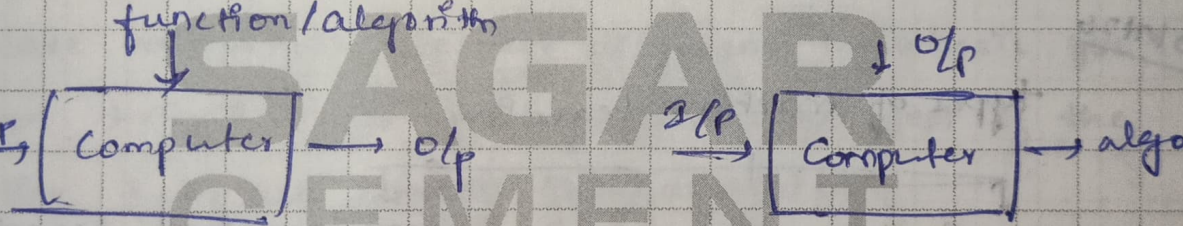
ML for Sensing Applications

→ Introduction
1) Linear regression & decision tree
2) Instance based learning
3) Probability and Bayes learning
4) Support Vector Machine (SVM)
5) Neural Network
6) Clustering and Computational Concept

Learning is the ability to increase one's behavior with experience
→ exploring algorithm to build model for classification / decision making / prediction

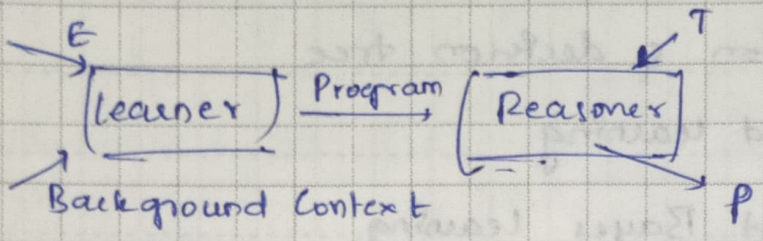


→ A computer program is said to learn from the experience "E" wrt some class of task 'T' and performance matrix "P" if its performance P on task T improves with experience E

E = data
T = classification / prediction / decision
P = accuracy

SAGAR ELEMENT

learner ⇒ training data
reasoner ⇒ test data.



ML architecture

→ Choose 'E' or training data
→ Choose a target function
→ how to represent your target function
→ Choose learning algorithm to infer the target function.

ML → E
→ T
→ P

30/01/26

types of machine learning

supervised learning — unsupervised learning

E ⇒ [x, y] → labelled data

$y = f(x)$

dataset has target feature 'y'
for input feature 'x'

where f → target function

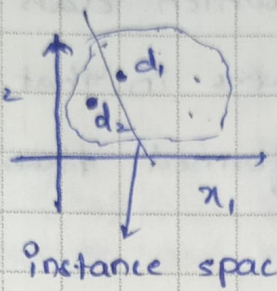
has instance ⇒ a set of information

let x_{in} are different features.

instance	X	Y
1	$x_{11}, x_{12}, \dots x_{1n}$	y_1
2	$x_{21}, x_{22}, \dots x_{2n}$	y_2
⋮		⋮
m	$x_{m1}, x_{m2}, \dots x_{mn}$	y_m

Instances can be plotted on feature space ≡ instance space

feature space →



instance space

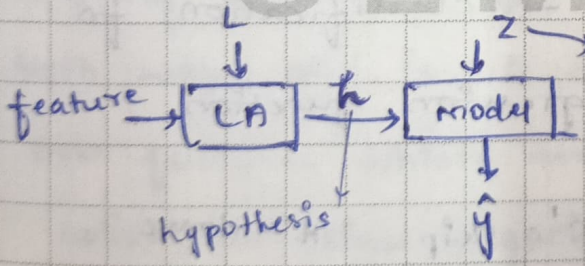
$d_1 : x_1 = 2, x_2 = 5$

$d_2 : x_1 = 1, x_2 = 3$

→ With a given language and feature set, there are a set of hypothesis.

→ All possible hypothesis (all legal target function) given ϵ, L forms hypothesis space

→ In case validation, we test different hypothesis for given L, ϵ but in testing phase, we test if the earlier L, ϵ are correct (how accurate)



→ Labelling done wrt ground truth

+

not always possible. ripeness of fruit, we get feature values but there isn't any ground truth to classify wrt it

But we can make clusters.

SAGAR CEMENT

Unsupervised learning is used where labelling is not so easy.

→ Semisupervised learning: Once the clusters are formed. few points can be used to identify which cluster belongs to which class and labelled for all others in that cluster

from the labels formed, target function is extracted

⇒ 4 categories of input feature

1) Categorical (A, AB, B, O)

2) Ordinal ⇒ can be measured but usually not and given classes ex: high, low, tall etc

3) Integer

4) Real

⇒ 2 types of o/p feature

1) discrete / integer → classification fn

2) continuous → regression function

K = known N = new S' = Skip H = Home

U = unknown O = old R' = Read W = work

R = Road L = long S = short

SAGAR CEMENT

Instance	Author	Thread	Action	Situation	Type
e_1	Known	New	Skip	H	Long
e_2	Unknown	N	S'	W	Short
e_3	U	O	Read	R	L
e_4	K	old	S'	W	L
e_5	K	N	R'	H	S
e_6	K	O	R'	H	L
	(x_1)	(x_2)	y	(x_3)	(x_4)

⇒ related to a person and reading an article

⇒ Inductive learning - we dont know all the instances

⇒ Hypothesis should be consistent & generalised

↓

should work with unseen data

testing data

⇒ features $x_1, x_2 \ldots$ may be important or not, so introduction of new feature or removal of existing feature might affect the accuracy in both ways and is lil difficult to determine

⇒ Few features which are not important are considered noise and since algorithm cant identify what are noise & what aren't we provide some "bias"

⇒ for $x_1 \mid x_2 \mid x_3 \mid x_4 \mid y$ ⇒ no.of interface = 2^N possible $= 16$

$F \Rightarrow S \subset X$

no.of target function $= 2^{2^N} = 2^{16}$. $H \rightarrow h \rightarrow f$

Bias → restriction
↘ preference

for S ⊂ X

S_1 S_2 S_3

↘ ↓ ↙ ⇒ If all training sets lead to same hypothesis h

then h is said to be consistent

& If h performs well with test instances, then it is generalised

test ∩ train data = ∅

Bias → Bias error ⇒ Condition imposed has caused some error ⇒ consistency issues

↘ variance ⇒ generalisation issues

underfitting ⇒ very simple h ⇒ missing nuances

↓ ⇒ caused by heavy bias

low training, low testing accuracies ⇒ variance ≈ low, B.E ≈ high

overfitting ⇒ high training accuracies
low testing

Bias error = low
variance = high

05/08/26

Given $\mathcal{H}, S \xrightarrow[B]{\text{choose}} h$ S ⇒ training set

how good is h? $\mathcal{H}$ ⇒ hypothesis space

$h(x) \rightarrow \hat{y}$

$e = (\hat{y} - y)$

accuracy, error, precision, recall, confusion matrix

SAGAR CEMENT

E, T, P

↓ dataset ↳ Language, L

Instance - x

↳ feature - feature space

$y = f(x)$ — $h \xrightarrow{\text{estimate}} f$

↳ concept — form H — ↳ target function

Test Sample ⇒ experience that was not used in training or learning

↳ unseen samples

poor consistency ⇒ Bias error ; poor generalisation ⇒ variance

absolute error = $|\hat{y} - y|$

sum of square error = $\sum_{i=1}^{N} (\hat{y}_i - y_i)^2$; N = no. of test samples

average absolute error = $\frac{1}{N} \sum_{i=1}^{N} |\hat{y}_i - y_i|$

RMS error = $\sqrt{\frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2}$

⇒ all these errors are for regression problem, classification are categorical & no numerical ~~outputs~~ errors.

for classification problems ⇒

avg no. of misclassification = $\frac{1}{N} \sum_{i=1}^{N} q_i(\hat{y}_i, y_i)$

where $q_i(\hat{y}, y) = 1$ if $\hat{y} = y$

$= 0$ o/w.

⇒ if correctly classified, $q_i(\hat{y}, y) = -$

confusion matrix :

hypothesised class \ True	P	N
P	TP	FP
N	FN	TN

P = positive,
N = negative
ex: colour yellowish or not

2-class classification problem

for three or more ⇒

H \ T	R	G	B
R	TP	FP/FN	FP/FN
G	FP/FN	TP	FP/FN
B	FP/FN	FP/FN	TP

→ depends on perspective whether we are considering wrt R or G or B

precision: $\frac{TP}{TP+FP} = \frac{TP}{h_P}$

$P_R = \frac{TP_R}{TP_R + FP_R}$

recall: $\frac{TP}{TP+FN}$; $R_R = \frac{TP_R}{TP_R + FN_R}$

precision ⇒ out of all samples which are identified as true, how many are actually true

recall ⇒ out of all true samples, how many were identified

ex: 10 yellow, 6 not

10 yellow → 8 yellow, 2 not
6 not → 3 yellow, 3 not

$P = \frac{8}{8+3}$

$P = \frac{8}{8+2}$

SAGAR CEMENT

recall ⇒ need to be maximum in medical applications

sample error ⇒ error for a sample of data.

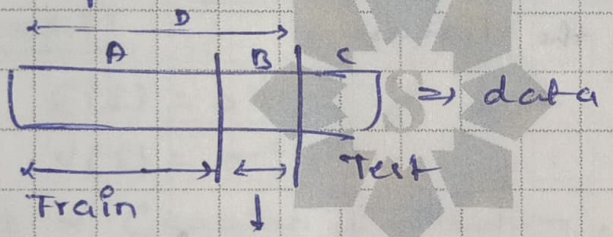
⇓ ↳ error for some instance / data ∈ Entire data

$$error_T(h) = \frac{1}{N} \sum_{x \in T} \delta(h(x), c(x))$$

True error ⇒ if for all such instances / data samples possible ∈ entire data, an error is found, it is true error ⇒ impossible to find.

$$error_D(h) \Rightarrow P_{x \in D}(h(x) \neq c(x))$$

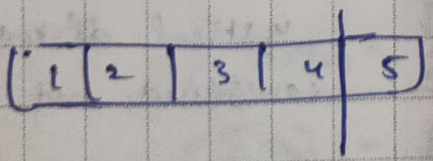
using sample error, we estimate true error



test data can't be used to tune coz it should remain unseen so train data is further split into train & validation sets

K-fold cross validation:

divide the entire train data D, into multiple sets, let say 5;



⇒ perform multiple iterations ⇒ 1st iter, 1234 ⇒ train, 5 ⇒ validation

2nd ⇒ 2345 - train, 1 ⇒ validation

3rd ⇒ 1345 - train, 2 ⇒ validation

⋮

06/08/26

Wheat	Septoria	} diseases
Maize	Blight	
Potato	rust	
Tomato	Healthy	
plant / crops	condition	

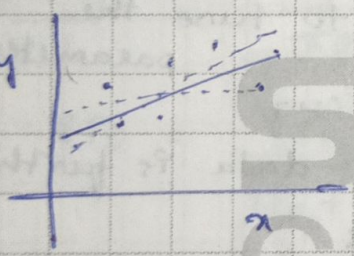
given ⇒ plant leaves cant be differentiated
but diseases can be

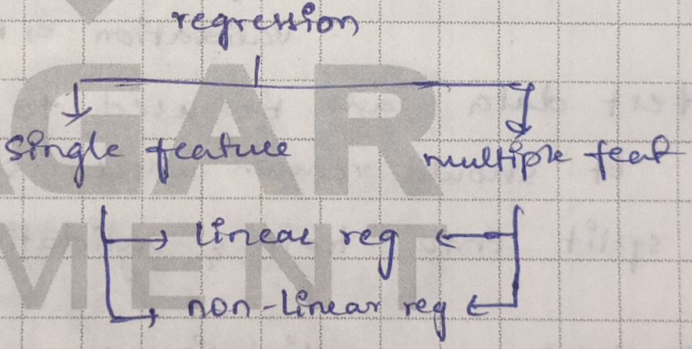
⇒ labels are available only for diseases

⇒ crop identification = unsupervised (semi
disease = supervised

⇒ crop identification forms clusters,
then identifies the disease.

→ Regression:

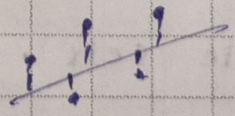




$x_1 [x_1, \ldots x_n]$

Sum of Square error of vertical distances from points to the line should be minimum

one of the methods to find the best fit



vertical distances are considered, not perpendicular

Fitting function / hypothesis, make it as simple as possible

SSD = minimum doesnt always give the best fit

⇒ scope side limitation

10/08/26

Bias → Assumption / Perception → Domain knowledge
Features / Noise → Perception

Construction of regression:

Single Feature linear regression:

β = weight / parameter / estimation

$$h(x) = \beta_0 + \beta_1 x$$

$$y_{(k)} = \beta_0 + \beta_1 x_{(k)} + \varepsilon_k$$

$$y_{(1)} = \beta_0 + \beta_1 x_{(1)} + \varepsilon_{(1)}$$
$$y_{(2)} = \beta_0 + \beta_1 x_{(2)} + \varepsilon_{(2)}$$
→ different training data

$\varepsilon_{(1)}$, $\varepsilon_{(2)}$ ⇒ noise models in different training data.

Noise model:

$$E(\varepsilon) = 0$$
$$V(\varepsilon) = \sigma^2$$

If pdf ⇒ Guassian distribution.
then noise model is white noise

SSE ⇒ $\theta = [\beta_0, \beta_1]$

$$J(\theta) = \frac{1}{2}[y - h(x)]^2$$
(single training point / example)

↳ cost function

analytical

$$\min J(\theta) = \min \sum_{d \in D} \frac{1}{2}\left[y_{(k)} - (\beta_0 + \beta_1 x_{(k)})\right]^2$$

D = entire dataset

$$\beta_1 = \frac{P\sum_{k=1}^{P} x_{(k)} y_{(k)} - \sum_{k=1}^{P} x_{(k)} \sum_{k=1}^{P} y_{(k)}}{P\sum_{k=1}^{P} x_{(k)}^2 - \left(\sum_{k=1}^{P} x_{(k)}\right)^2}$$

Co-variance

$$\beta_0 = \frac{\sum_{k=1}^{P} y_{(k)} - \beta_1 \sum_{k=1}^{P} x_{(k)}}{P}$$

P ⇒ no. of training samples.

Multifeature linear regression:

$[\bar{x}] = x_1, x_2, \ldots x_N$

N = no. of features

$$h(x) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots \beta_N x_N$$

$$= \sum_{j=1}^{N} \beta_j x_j + \beta_0$$

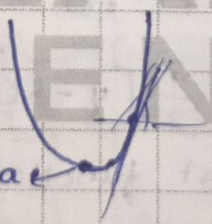
⇒ need 'N+1' no. of estimators for actual target fn 'N' no. of predictor variable

↳ To find analytically, involves N×N matrix inversion hence we heavily rely on iterative method which update β_i on each iteration

LMS method → least minimum slope:

cost function: J(θ) → quadratic function
↳ convex function

⇒ delta rule



only one minima

⇒ single training point:

$$J(\theta) = \frac{1}{2} \sum_{i=0}^{m} \left[h(\bar{x})^i - y^{(i)} \right]^2$$

find θ that minimise J(θ)

⇒ start search with initial guess of $\theta[\beta_0^\circ, \beta_1^\circ, \ldots]$ and update with delta rule

SAGAR CEMENT

$$\frac{\partial}{\partial \theta} J(\theta) \Rightarrow \frac{\partial}{\partial \theta_j}\left(\frac{1}{2}\left[h(\bar{x}) - y\right]^2\right)$$

$$= \left[h(\bar{x}) - y\right] \frac{\partial}{\partial \theta_j}\left(h(\bar{x}) - y\right)$$

$$= \left[h(\bar{x}) - y\right] x_j$$

single training point -

LMS update rule : $\beta_j^{(i+1)} = \beta_j^{(i)} + \alpha\left(y^{(i)} - h(\bar{x}^{(i)})\right) x_j^{(i)}$

incremental / stocastic G.D ←

if $> 0 \Rightarrow \beta_j \uparrow$

$< 0 \Rightarrow \beta_j \downarrow$

$= 0 \Rightarrow$ no change

p training examples -

$$\beta_j^{(i+1)} = \beta_j^{(i)} + \alpha \sum_{k=1}^{p} \left[y_{(k)}^{(i)} - h\left(\bar{x}_{(k)}^{(i)}\right)\right] x_{j(k)}$$

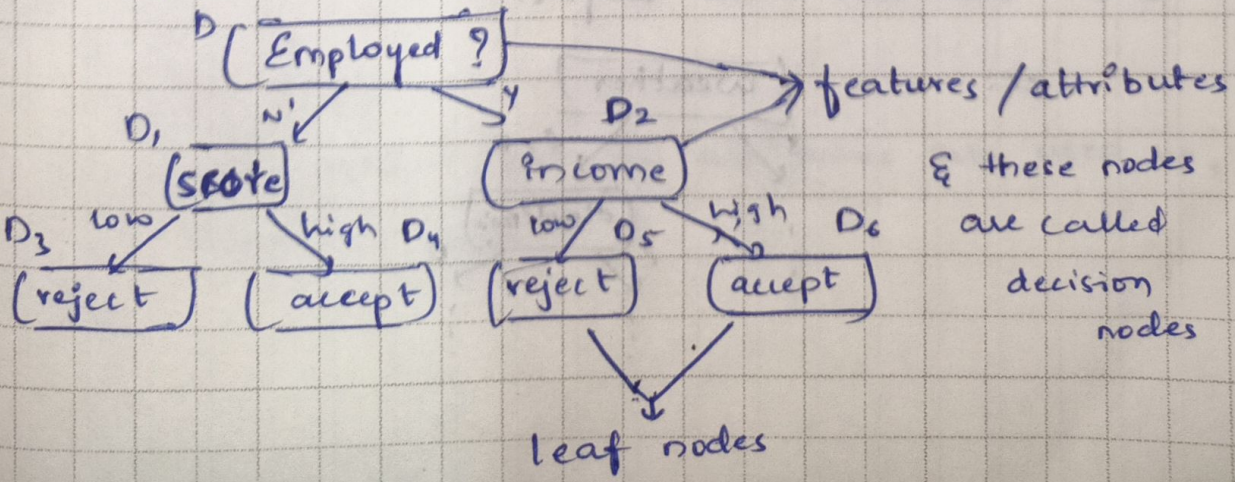
- Batch gradient descent (mini batch G.D)

19/08/26

Decision Tree

- Tree structured classification
- Can also work for regression
- Boolean function which is disjunction of conjunction

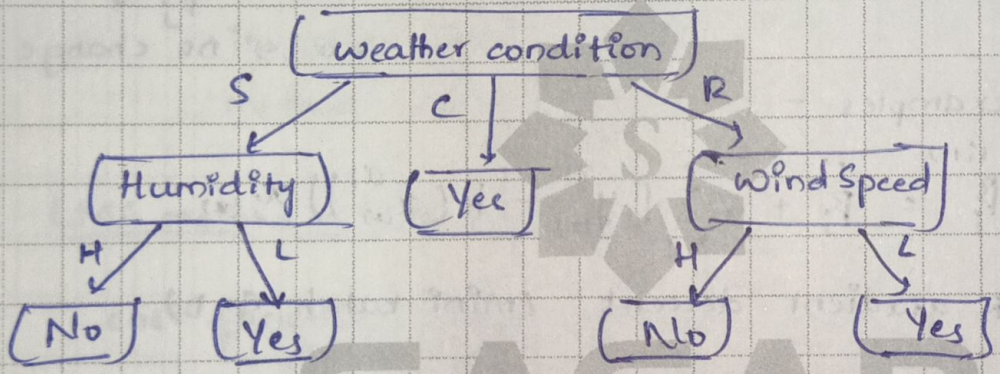
Ex: Target Concept = Approval of Loan



Multi variant decision tree

Ex: Target Concept = Shall we play tennis today.

Attribute	Values
weather	cloudy, sunny, rainy
temperature	hot, cold
humidity	high, low
wind speed	high, low
Target	Yes, No

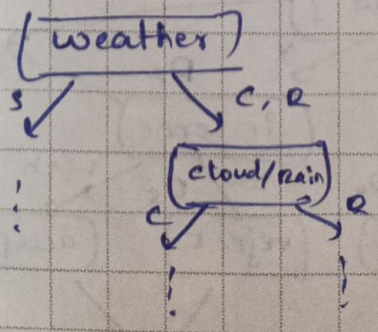


test instance ⇒ Sunny day, warm, low humidity & high wind speed.

⇒ Temperature attribute is not being used at all, can be considered noise

⇒ Multivariat d.t can be modelled as too variat d.t but has more layers.

ex:



SAGAR CEMENT

Bias : assuming our target fn is a decision tree

preference : Simple d.t → lower depth
↳ lower no. of decision nodes

Construct of a decision tree:

1) Identify an attribute 'A'

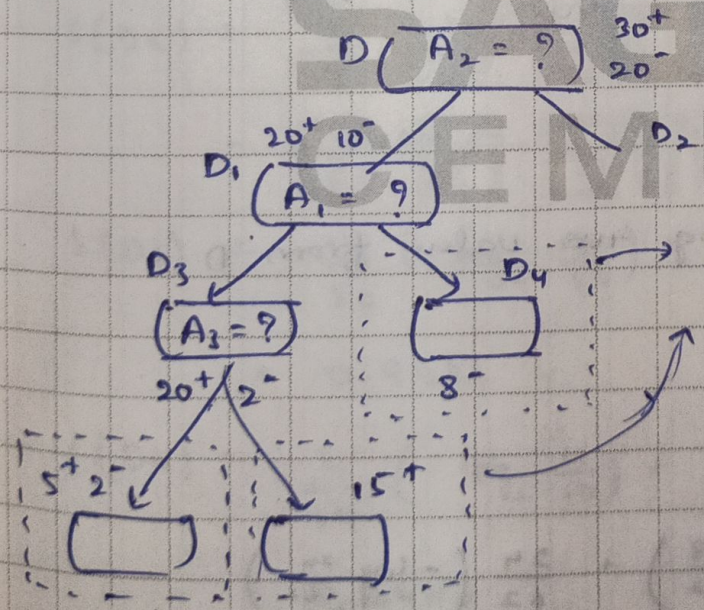
2) Based on values of A, split 'D'

3) Based on the split, decide to stop or continue, if to continue, what is next 'A'

Q)

→ When to stop ⇒ a) When all attributes are used up
b) when one split has all examples of same category
c) When very few e.g are left

Ex: A_1, A_2, A_3

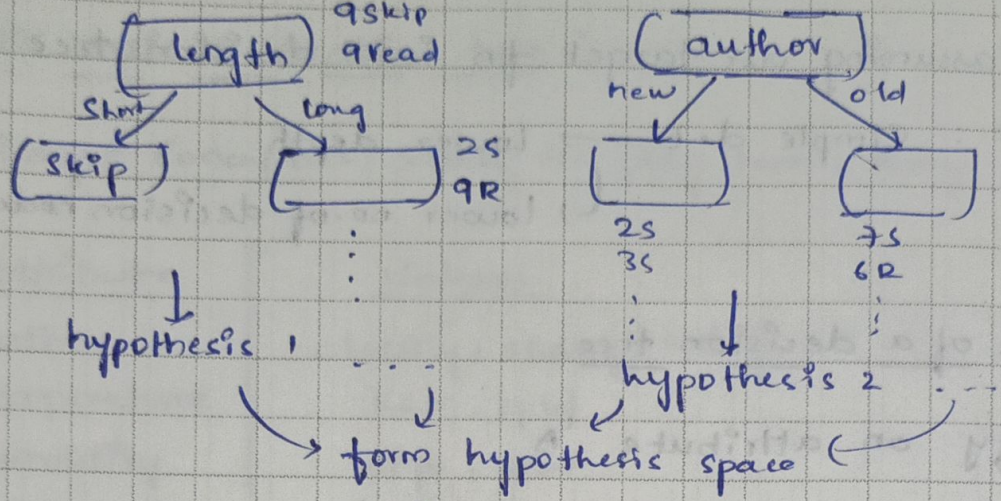


→ No further split needed cuz all are of same category/type

↳ split needed but all attributes are used up, have to stop.

SAGAR CEMENT

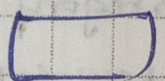
Ex:



Entropy: measure of disorder in a system

$$\text{Entropy}(D) = S = P_+(-\log_2 P_+) + P_-(-\log_2 P_-) \quad \text{—(1)}$$

in case indecisive nodes, which cannot be further



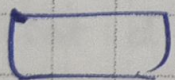
2+ (50%) split, the probability of both values affect the accuracy for every run

→ even with same data, results can vary due to these probabilites

P_+ = probability of attaining +ve value from D

P_- = for −ve

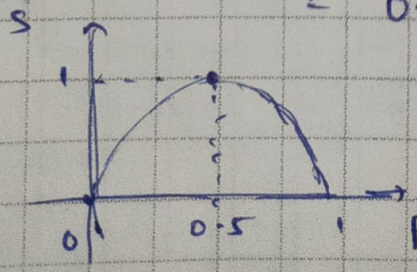
Ex:



$D = 38^+$
$= 24^-$

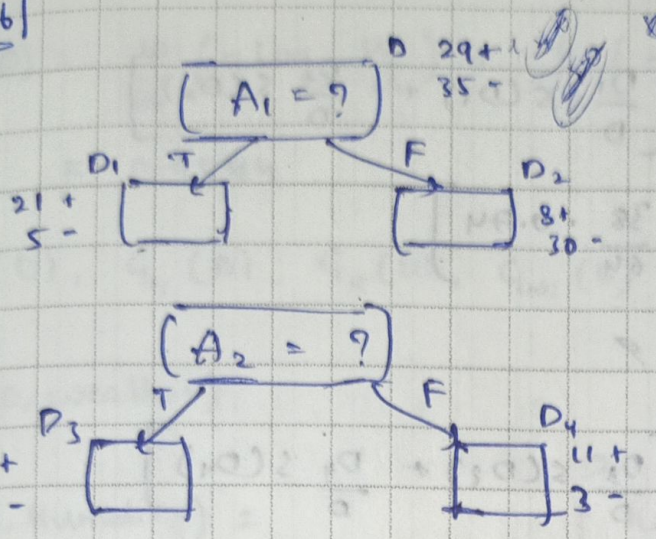
$$E(D) = \frac{38}{62}\left(-\log_2 \frac{38}{62}\right) + \frac{24}{62}\left(-\log_2 \frac{24}{62}\right)$$

$$= 0.96$$



SAGAR CEMENT

20/08/26



entropy $S(D) = P_+(-\log_2 P_+) + P_-(-\log_2 P_-)$

$$S(D) = \frac{29}{64}\left(-\log_2 \frac{29}{64}\right) + \frac{35}{64}\left(-\log_2 \frac{35}{64}\right)$$

$$= 0.9936$$

$$S(D_1) = \frac{21}{26}\left(-\log_2 \frac{21}{26}\right) + \frac{5}{26}\left(-\log_2 \frac{5}{26}\right)$$

$$= 0.7062$$

$$S(D_2) = \frac{8}{38}\left(-\log_2 \frac{8}{38}\right) + \frac{30}{38}\left(-\log_2 \frac{30}{38}\right)$$

$$= 0.7424$$

$$S(D_3) = \frac{18}{50}\left(-\log_2 \frac{18}{50}\right) + \frac{32}{50}\left(-\log_2 \frac{32}{50}\right)$$

$$= 0.9426$$

$$S(D_4) = \frac{11}{14}\left(-\log_2 \frac{11}{14}\right) + \frac{3}{14}\left(-\log_2 \frac{3}{14}\right)$$

$$= 0.7496$$

Gain in information

$$\text{Gain}(D, A) = S(D) - \sum_{v \in \text{values}(A)} \frac{|D_v|}{|D|} \cdot S(D_v)$$

SAGAR CEMENT

$$Gain(D, A_1) = S(D) - \left[\frac{D_1}{D} \cdot S(D_1) + \frac{D_2}{D} S(D_2)\right]$$

$$0.99 - \left[\frac{26}{64} \cdot 0.71 + \frac{38}{64} \cdot 0.74\right]$$

$$Gain(D, A_1) = 0.2621$$

$$Gain(D, A_2) = S(D) - \left[\frac{D_3}{D} S(D_3) + \frac{D_4}{D} S(D_4)\right]$$

$$= 0.99 - \left[\frac{50}{64} \cdot 0.94 + \frac{14}{64} \cdot 0.75\right]$$

$$= 0.0915$$

Instances	Weather	Temp	Humid	Wind	Action
1	S	H	H	W	X
2	S	H	H	S	X
3	C	H	H	W	✓
4	R	M	H	W	✓
5	R	C	N	W	✓
6	R	C	N	S	X
7	C	C	N	S	✓
8	S	M	H	W	X
9	S	C	N	W	✓
10	R	M	N	W	✓
11	S	M	N	S	✓
12	C	M	H	S	✓
13	C	H	N	W	✓
14	R	M	H	S	X
15	S	C	H	W	X
16	C	C	N	W	✓

$S(D) = ?$ $\quad D = 6\ (-)$
$\quad\quad\quad\quad\quad\quad 10\ (+)$

SAGAR CEMENT

$$S(D) = \frac{10}{16}\left(-\log_2 \frac{10}{16}\right) + \frac{6}{16}\left(-\log_2 \frac{6}{16}\right)$$

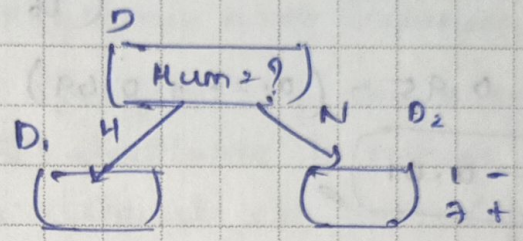
$$= 0.9544$$

$G_w(D)$, $G_T(D)$, $G_H(D)$, $G_{Wi}(D)$

↓

$G(D, \text{weather})$

$G(D, \text{Humidity}) =$



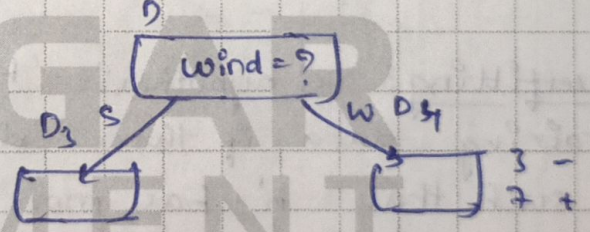
$$S(D) - \left[\frac{8}{16} S(D_1) + \frac{8}{16} S(D_2)\right]$$

$$S(D) - 0.5\left(\frac{3}{8}\left(-\log_2 \frac{3}{8}\right) + \frac{5}{8}\left(-\log_2 \frac{5}{8}\right) + \frac{1}{7}\left(-\log_2 \frac{1}{7}\right) + \frac{7}{8}\left(-\log_2 \frac{7}{8}\right)\right)$$

$$= 0.95 - 0.7489$$

$$G_H = 0.2011$$

$G(D, \text{wind}) =$



$S(D_3) = 1$

$S(D_4) =$

$$-\frac{3}{10}\log_2\left(\frac{3}{10}\right) + \frac{7}{10}\left(-\log_2 \frac{7}{10}\right)$$

$$= 0.8812$$

$$\text{Gain}(D, \text{wind}) = 0.95 - \left(\frac{6}{16} \times 1 + \frac{10}{16} \times 0.8812\right)$$

$$G_{wi} = 0.02425$$

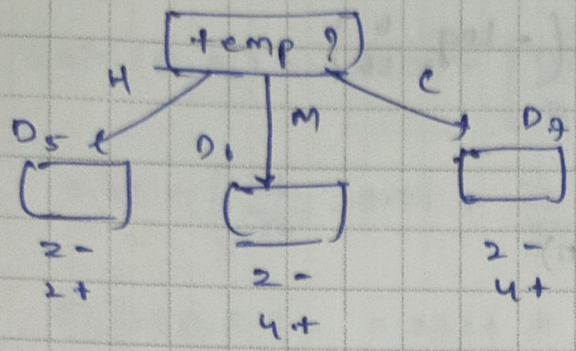
zone out

Gini index $= 1 - \sum_{i=1}^{n} (P_i)^2$

entropy $= -\sum_{i=1}^{n} P_i \log_2 (P_i)$

SAGAR CEMENT

zone back in



$S(D_5) = 1$

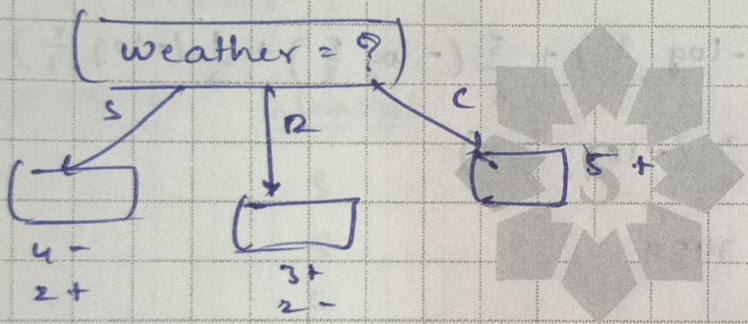
$S(D_1) = 0.92$

$S(D_9) = 0.92$

$$\text{Gain}(D, \text{temp}) = 0.95 - \left(\frac{4}{16} \times 1 + \frac{12}{16} \times 0.92\right)$$

$$\simeq 0.95 - (0.25 + 0.69)$$

$$\boxed{G_t = 0.01}$$

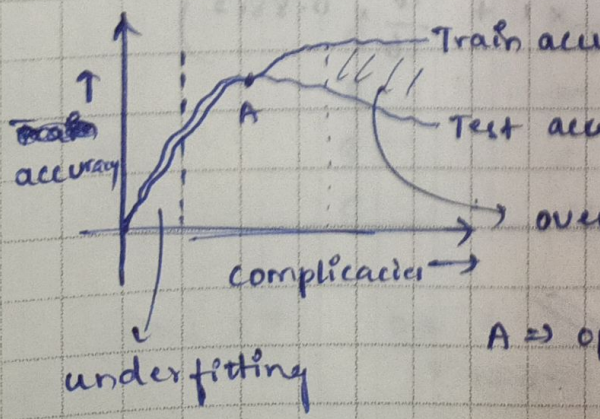


27/08

Overfitting: - A hypothesis 'h' is said to overfit a training data if there exists another hypothesis 'h' such that h' has more error than h on training data but h' has less error than h on test data / unseen test set

Complicacies ⇒ d.T = no. of nodes

regression = order of polynomial



A ⇒ optimal point

pruning → pre pruning ⇒ Early stop
↳ post pruning

↓

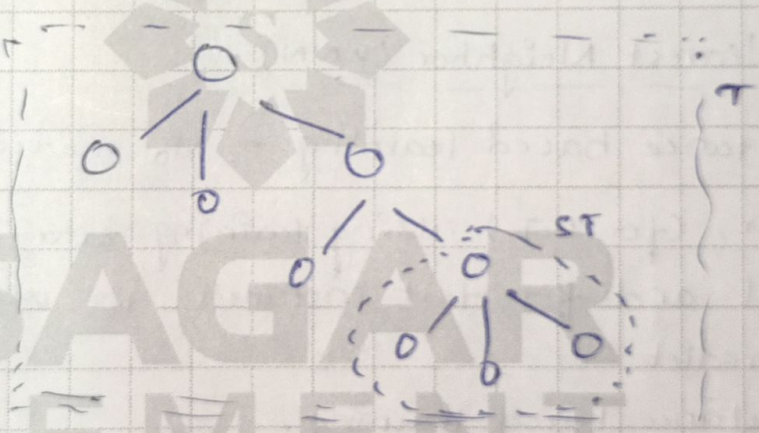
scope for further splitting of a node but we want to avoid overfitting : i.e., to avoid increase in complexity

Early stop can happen → i) When no. of instances is less than some threshold specified by the user

ii) Available instances are uncorrelated with the residual attributes (features)

iii) No increase in information gain or less

post pruning ⇒



Check if T or T-ST has less error

Training data → Grow set ⇒ entire tree developed using this
↳ validation set

↓

Check if T has less error or T-ST has less error on validation set

↓

used for hyperparameter tuning of the model, never to measure the true accuracy of the model

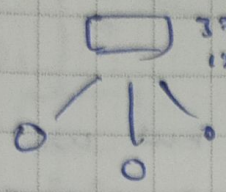
True accuracy

↓

Test data is used for this

SAGAR CEMENT

T - ST : replace the ST part with majority value

ex : 

⇒ replace entire node with '+'

if T - ST has less error, freeze it

1) Complexity of h ⇒ Comply ↑ = train error ↓
test error ↓, ↑

2) Training data size ⇒ N ↑ = train error ↓
test error ↓

Generalisation ⇒ test error ↓

07/09

K-Nearest Neighbor (KNN)

↳ instance based learning ⇒ No model is made

⇒ earlier, (for D.T) using training data, we develop a model and for test instances, we use this model to predict

⇒ In instance based learning,
during training phase ⇒ store training data
during testing phase ⇒ prediction time → get a test instance
→ forming the model

instance based learning —AKA→ Lazy learning

↓

Not good for large feature size ⟶ Feature selection / feature dimension reduction is needed.

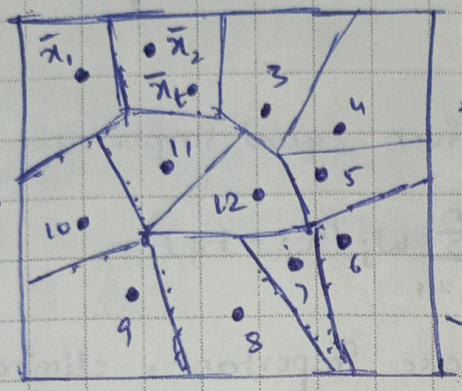
⇒ When we get a test instance, we check which training instances are similar / close to this test instance.

SAGAR CEMENT

→ similarity / closeness can be measured using distance b/w points when represented geometrically.

↓ Euclidean distance.

Ex:



⇒ find the bisectors of two adjacent training pts & form the decision boundary

→ Voronoi diagram for 1-NN

$\bar{x}_1 = [x_{1a}, x_{1b}] \rightarrow y_1$
$\bar{x}_2 = [x_{2a}, x_{2b}] \rightarrow y_2$

represent something, like red / blue / green etc in case of color classification

let y_1, y_2, y_3, y_4, y_8 = red

rest ⇒ green

* Instance based learning is highly sensitive to output noise.

1, 2, 3, 4 ⇒ closer to each other,

8 ⇒ also red ⇒ due to noise → i) Noise in attribute
ii) Noise in output label

Moving Average ⇒ can eliminate noise

⇕ similar to

low pass filter in electrical domain

⇒ Instead of just one nearest neighbor, we choose more = k-NN

↳ what should 'k' be ?

Euclidean distance $= D(x_i, x_t) = \sqrt{\sum_{j=1}^{n} (x_{ij} - x_{tj})^2}$

x_i = some training instance

x_t = test instance

n = total no. of features

⇒ Out of all the 'k' closest neighbors, choose the majority class in case of classification

⇒ In case of regression, take the average of all those k-neighbors.

⇒ All features should not be given equal importance.

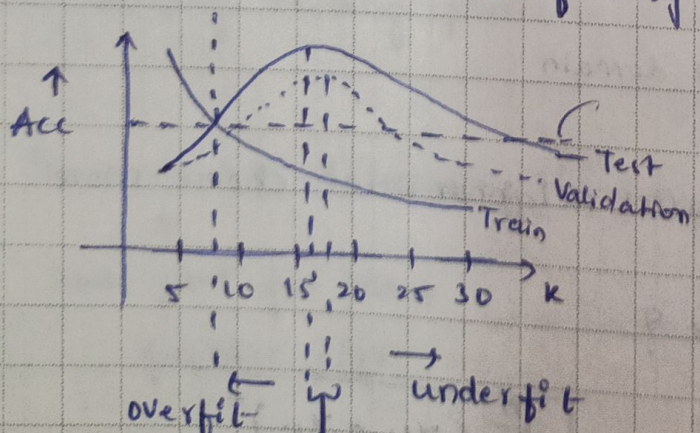
∴ weighted Euclidian distance $= \sqrt{\sum_{j=1}^{n} w_j (x_{ij} - t_{ij})^2}$

⇒ To find out which features are important, eliminate the ones which are prone to noise

↳ Correlation coefficient,
PCA, LDA, t-SNE etc can be used

⇒ Ambigous labelling can also introduce noise
i.e: fruit ripening is a continuous process
no fixed labels.

⇒ In case of D.T, more nodes ⇒ more complicate
kNN, less k ⇒ more complicate
if k is less ⇒ more bounding conditions
↳ overfitting



09/09

Small k

→ Capture structure of a problem space with fine boundaries
→ Susceptible to overfitting
→ Need to apply if training set is small

large k

→ Yield more coarse boundary
→ less sensitive to noise
→ Better probabilistic estimator
→ Susceptible to underfitting if k is too large
→ Needs larger training set

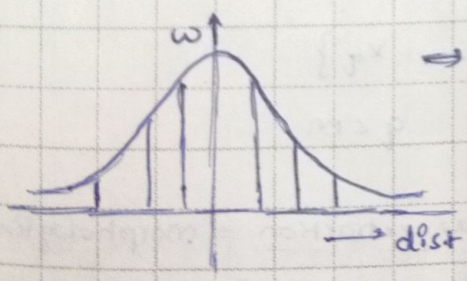
Equal weights

① Spherical attributes → comparable σ
② Normalized range (similar scales)

Distance weighted kNN:

⇒ Trade off between small and large k - difficult
→ use large k - give more emphasis on nearest neighbour

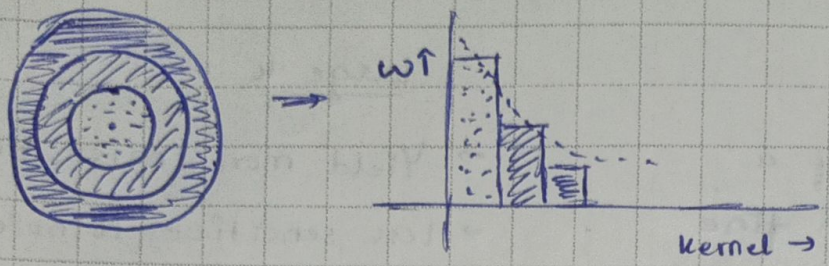
$$w_i = \frac{1}{Dist(x_i, x_t)}$$ ⇒ inverse distance weightage



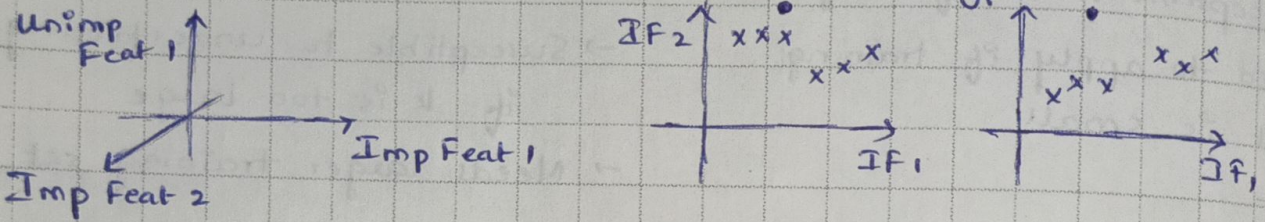
⇒ Gaussian weightage function
(locally weighted averaging)

weighted kernel:

$$w_i = e^{\frac{1}{k.w \times Dist(x_i, x_t)}}$$; k.w = kernel width

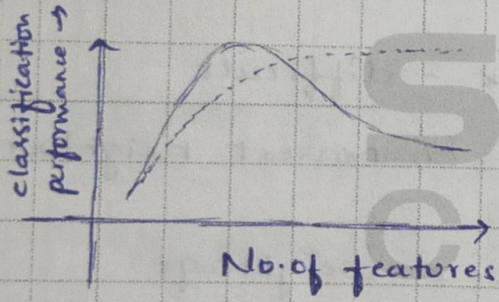


Why feature selection ?



more features → more information
↳ Better discriminative power } not always true

1) Irrelevant features → Noise
2) Redundant features → unnecessary weightage
↓
Small train set
↓
More error



Feature extraction

$\{x_1, x_2, \ldots x_m\} \longrightarrow \{x_1, x_2, \ldots x_q\}$

$q < m$

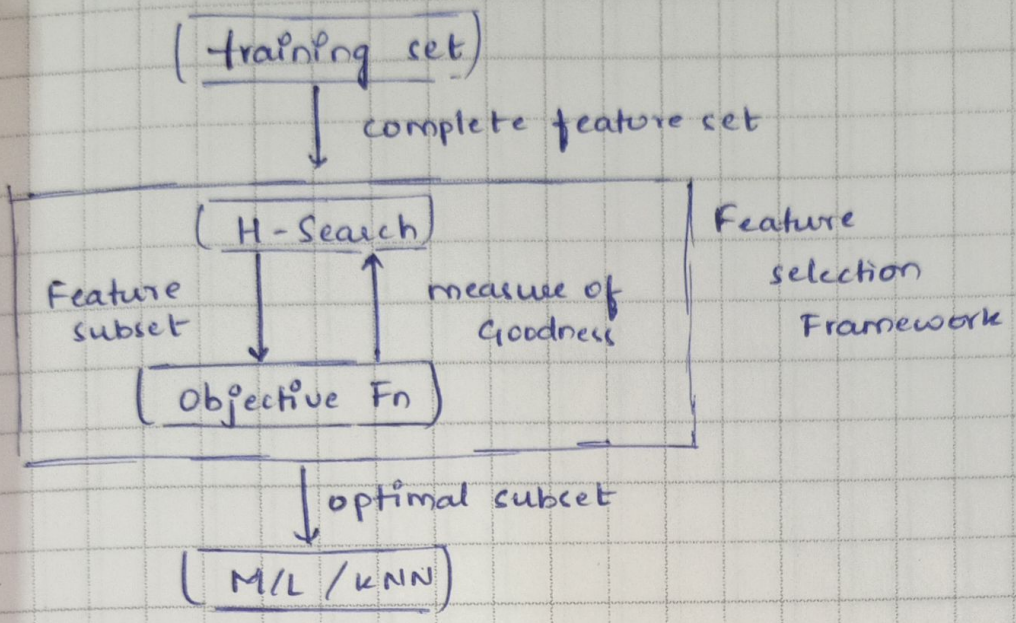
↓ Feature selection

$\{x_1', x_2', \ldots x_p'\}$ $p < m$

Feature extraction = morphological change that gives out some completely new features

Feature selection ⇒ 2^m possibs

→ optimisation ⇒ Feats need to be mathematically associated together
→ Heuristic method
→ Random method ⇒ domain knowledge



Goodness → wrapper method (Supervised) ⇒ Train grow set with subset and validate with validation set

→ Filter method (unsupervised) ⇒ Evaluate statistical into embedded

wrapper method → Forward method

→ Backward "

Forward ⇒

1 - 67 %
2 - 71 %
3 - 69 %
4 - 84 %
5 - 65 %
6 - 41 %

→

4,1 → 87 %
4,2 → 63 %
4,3 → 51 %
4,5 → 24 %
4,1 → 61 %.

→ ...

Backward ⇒